Development & Implementation Plan

This document provides a stepwise roadmap for building, testing, and deploying the AI Interview Room platform with its multimodal anti-cheating engine.

Phase 1: Prototyping & Foundation (Months 1-2)

Objective: Establish core infrastructure and validate the feasibility of real-time computer vision in the browser.

- **Architecture Setup:** Deploy PostgreSQL, MongoDB, and Redis. Initialize the Node.js API Gateway.
- **Frontend WebRTC:** Build the React interface capable of capturing webcam/microphone data and establishing WebRTC peer connections.
- **CV Prototype (MediaPipe):** Integrate MediaPipe into a Python microservice. Achieve basic extraction of 468 facial landmarks and 21 hand keypoints from a video stream.
- **Milestone 1:** A functional browser client that streams video to the backend and logs basic head pose and hand coordinates.

Phase 2: Anti-Cheating Core AI Integration (Months 3-4)

Objective: Develop the detection algorithms and threshold logic.

- **Gaze & Eye Tracking:** Implement the mathematical logic to calculate gaze deviation vectors. Set up the 25-degree threshold and temporal smoothing to prevent flickering false positives.
- **Hand & Object Detection:** Write algorithms to detect "hand out of frame" boundaries (>80%).
- **Audio & Deepfake Pipeline:** Integrate Whisper for real-time STT. Build the challenge-response randomized prompt logic for the liveness check.
- **Milestone 2:** The backend successfully generates time-series telemetry logs in MongoDB identifying cheating events in real-time.

Phase 3: Conversational AI & Cognitive Fingerprinting (Months 5-6)

Objective: Make the AI Interviewer smart and implement behavioral biometrics.

- **LLM Integration:** Connect an LLM (e.g., GPT-4 or Claude) to act as the interviewer, parsing STT data and generating TTS responses.
- **Adaptive Questioning Logic:** Program the LLM to recognize textbook answers and pivot to highly specific, contextual follow-up questions.
- **Keystroke/Mouse Tracking:** Implement frontend event listeners to capture dwell/flight times and stream them via WebSockets to the backend for cognitive fingerprinting analysis.
- **Milestone 3:** A fully interactive AI interview session that actively probes suspicious answers and tracks typing behavior.

Phase 4: Recruiter Dashboard & Beta Testing (Months 7-8)

Objective: Build the analytics interface and validate the system with real users.

- **UI/UX Implementation:** Develop the Recruiter dashboard with video playback and the forensic timeline scrubber.
- **Scoring Algorithm:** Finalize the weights for the "Cheating Probability Score" (e.g., Gaze deviation = 40%, Audio anomalies = 30%, Hand flags = 30%).
- **Closed Beta:** Run a pilot program with a small group of friendly beta testers. Actively monitor false positive rates and adjust thresholds (especially for neurodivergent edge cases).

- **Milestone 4:** System is usable end-to-end by beta recruiters.

Phase 5: Production Deployment & Global Scaling (Month 9-10)

Objective: Security hardening, global compliance auditing, and multi-region launch.

- **Security & Sovereignty Audit:** Conduct external penetration testing and SOC 2 Type II compliance reviews. Ensure data purging cron jobs and EU/LATAM data residency shards are functioning correctly (GDPR/LGPD).
- **Global Infrastructure Scaling:** Implement Kubernetes (K8s) auto-scaling for the Python CV inference nodes deployed across AWS G5 clusters in US-East, EU-Central, and AP-South. Configure Cloudflare/AWS Global Accelerator for Edge WebRTC routing.
- **Hyper-Polyglot Rollout:** Activate Whisper and LLM prompts across 50+ localized languages. Enable live real-time dashboard translations for recruiters.
- **Launch:** Deploy to global production.

Risk Management & Contingency

- **Risk: High False Positive Rates (e.g., candidate looking at a physical notepad).**
  - *Mitigation:* The system does not automatically fail candidates. It flags events for human review. Provide clear instructions to candidates about allowed behaviors.
- **Risk: CV Inference Latency lagging the video stream.**
  - *Mitigation:* Implement frame skipping (process every 3rd frame). Utilize dedicated GPU instances (NVIDIA T4/A10G) for inference.
- **Risk: Evasion by new Deepfake techniques.**
  - *Mitigation:* Continuously update the Challenge-Response library so the required physical gestures cannot be pre-recorded or easily mapped by RTDF pipelines.